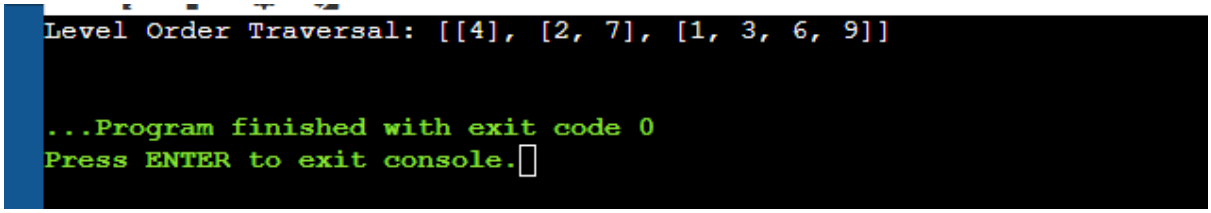
```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int val;
    struct TreeNode *left;
    struct TreeNode *right;
};

int** levelOrder(struct TreeNode* root,
                 int* returnSize,
                 int** returnColumnSizes) {

    if (root == NULL) {
        *returnSize = 0;
        *returnColumnSizes = NULL;
        return NULL;
    }

    int nodeCount = 0;

    struct TreeNode** countStack = malloc(10000 * sizeof(struct TreeNode*));
    int countTop = 0;

    countStack[countTop++] = root;

    while (countTop > 0) {
        struct TreeNode* current = countStack[--countTop];
```

```
    nodeCount++;

    if (current->left != NULL)
        countStack[countTop++] = current->left;

    if (current->right != NULL)
        countStack[countTop++] = current->right;
}

free(countStack);

struct TreeNode** queue =
    malloc(nodeCount * sizeof(struct TreeNode*));

int** result = malloc(nodeCount * sizeof(int*));

int* columnSizes = malloc(nodeCount * sizeof(int));

int front = 0;
int rear = 0;
int levels = 0;

queue[rear++] = root;

while (front < rear) {

    int size = rear - front;
```

```
        result[levels] = malloc(size * sizeof(int));

        columnSizes[levels] = size;

        for (int i = 0; i < size; i++) {

            struct TreeNode* current = queue[front++];

            result[levels][i] = current->val;

            if (current->left != NULL)
                queue[rear++] = current->left;

            if (current->right != NULL)
                queue[rear++] = current->right;
        }

        levels++;
    }

    free(queue);

    *returnSize = levels;
    *returnColumnSizes = columnSizes;

    return result;
}
```

```
int main() {

    struct TreeNode n1 = {1, NULL, NULL};
    struct TreeNode n3 = {3, NULL, NULL};
    struct TreeNode n6 = {6, NULL, NULL};
    struct TreeNode n9 = {9, NULL, NULL};

    struct TreeNode n2 = {2, &n1, &n3};
    struct TreeNode n7 = {7, &n6, &n9};

    struct TreeNode root = {4, &n2, &n7};

    int returnSize;
    int* columnSizes;

    int** result = levelOrder(&root, &returnSize, &columnSizes);

    printf("Level Order Traversal: [");

    for (int i = 0; i < returnSize; i++) {

        printf("[");

        for (int j = 0; j < columnSizes[i]; j++) {

            printf("%d", result[i][j]);

            if (j < columnSizes[i] - 1)
```

```
            printf(", ");
        }

        printf("]");

        if (i < returnSize - 1)
            printf(", ");

        free(result[i]);
    }

    printf("]\n");

    free(result);
    free(columnSizes);

    return 0;
}
```